# CS1312
# Algorithms Design and Analysis

## Fundamentals of Algorithms Analysis

# Analysis of Algorithms

- An *algorithm* is a finite set of precise instructions for performing a computation or for solving a problem.
- What is the goal of analysis of algorithms?
  - To compare algorithms mainly in terms of running time but also in terms of other factors (e.g., memory requirements, programmer's effort etc.)
- What do we mean by running time analysis?
  - **Determine how running time increases as the size of the problem increases.**

# Input Size

- Input size (number of elements in the input)
  - size of an array
  - polynomial degree
  - # of elements in a matrix
  - # of bits in the binary representation of the input
  - vertices and edges in a graph

# Types of Analysis

- **Worst case**
  - Provides an upper bound on running time
  - An absolute guarantee that the algorithm would not run longer, no matter what the inputs are
- **Best case**
  - Provides a lower bound on running time
  - Input is the one for which the algorithm runs the fastest

$$Lower\ Bound \leq Running\ Time \leq Upper\ Bound$$

- **Average case**
  - Provides a prediction about the running time
  - Assumes that the input is random

# How do we compare algorithms?

- We need to define a number of objective measures.

  (1) Compare execution times?

  ***Not good***: times are specific to a particular computer !!

  (2) Count the number of statements executed?

  ***Not good***: number of statements vary with the programming language as well as the style of the individual programmer.

# Ideal Solution

- Express running time as a function of the input size $n$ (i.e., $f(n)$).
- Compare different functions corresponding to running times.
- Such an analysis is independent of machine time, programming style, etc.

# Example

- Associate a "cost" with each statement.
- Find the "total cost" by finding the total number of times each statement is executed.

***Algorithm 1***

| | Cost | Times |
|---|---|---|
| arr[0] = 0; | $c_1$ | 1 |
| arr[1] = 0; | $c_1$ | 1 |
| arr[2] = 0; | $c_1$ | 1 |
| ... | ... | |
| arr[N-1] = 0; | $c_1$ | 1 |

$f(N) = c_1 + c_1 + \ldots + c_1 = c_1 \times N$

***Algorithm 2***

| | Cost | Times |
|---|---|---|
| for(i=0; i<N; i++) | $c_2$ | N+1 |
| arr[i] = 0; | $c_1$ | N |

$f(N) = (N+1) \times c_2 + N \times c_1 = (c_2 + c_1) \times N + c_2$

# Another Example

| **Algorithm 3** | **Cost** | **Times** |
|---|---|---|
| sum = 0; | $c_1$ | 1 |
| for(i=0; i<N; i++) | $c_2$ | N+1 |
| for(j=0; j<N; j++) | $c_2$ | N x (N+1) |
| sum += arr[i][j]; | $c_3$ | N x N |
| ---------- | | |

$f(N) = c_1 + c_2 \times (N+1) + c_2 \times N \times (N+1) + c_3 \times N^2$

# Asymptotic Analysis

- To compare two algorithms with running times *f(n)* and *g(n),* we need a **rough measure** that characterizes **how fast each function grows.**
- *Hint:* use *rate of growth*
- Compare functions in the limit, that is, **asymptotically!**

  (i.e., for large values of *n*)

# Asymptotic Notation

- O notation: asymptotic "less than":
  - f(n)=O(g(n)) implies: f(n) "≤" g(n)
- Ω notation: asymptotic "greater than":
  - f(n)= Ω (g(n)) implies: f(n) "≥" g(n)
- Θ notation: asymptotic "equality":
  - f(n)= Θ (g(n)) implies: f(n) "=" g(n)

# Rate of Growth

- Consider the example of buying *elephants* and *goldfish:*

  **Cost**: cost_of_elephants + cost_of_goldfish

  **Cost** ~ cost_of_elephants (approximation)

- The low order terms in a function are relatively insignificant for **large** $n$

$$n^4 + 100n^2 + 10n + 50 \quad \sim \quad n^4$$

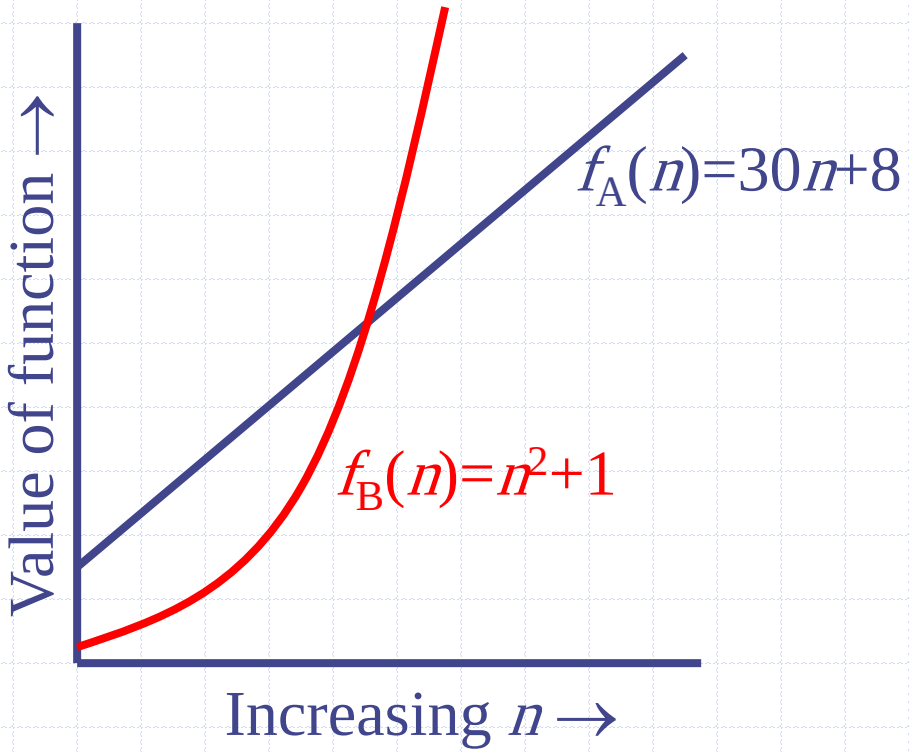
*i.e.,* we say that $n^4 + 100n^2 + 10n + 50$ and $n^4$ have the same **rate of growth.**

# Rate of Growth

- We say $f_A(n)=30n+8$ is *order* $n$, or O (n) It is, at most, roughly *proportional* to $n$.
- $f_B(n)=n^2+1$ is *order* $n^2$, or $O(n^2)$. It is, at most, roughly proportional to $n^2$.
- In general, any $O(n^2)$ function is faster- growing than any $O(n)$ function.

# Visualizing Orders of Growth

- On a graph, as you go to the right, a faster growing function eventually becomes larger...



# More Examples ...

- $n^4 + 100n^2 + 10n + 50$ is $O(n^4)$
- $10n^3 + 2n^2$ is $O(n^3)$
- $n^3 - n^2$ is $O(n^3)$
- constants
  - 10 is $O(1)$
  - 1273 is $O(1)$

# More Examples ...

- Calculate the big-O notation for:
  - f(n) = n [(n+1)/2]

Solution:

½ $n^2$ + ½ n (Remove the coefficient)

$n^2$ + n (keep the largest term)

$n^2$

So, the big-O notation is stated as O(*f(n)*)=O($n^2$)

# Back to Our Example

***Algorithm 1***

| | **Cost** |
|---|---|
| arr[0] = 0; | $c_1$ |
| arr[1] = 0; | $c_1$ |
| arr[2] = 0; | $c_1$ |
| ... | |
| arr[N-1] = 0; | $c_1$ |

$c_1+c_1+...+c_1 = c_1 \times N$

***Algorithm 2***

| | **Cost** |
|---|---|
| for(i=0; i<N; i++) | $c_2$ |
| arr[i] = 0; | $c_1$ |

$(N+1) \times c_2 + N \times c_1 = (c_2 + c_1) \times N + c_2$

- Both algorithms are of the same order: *O(N)*

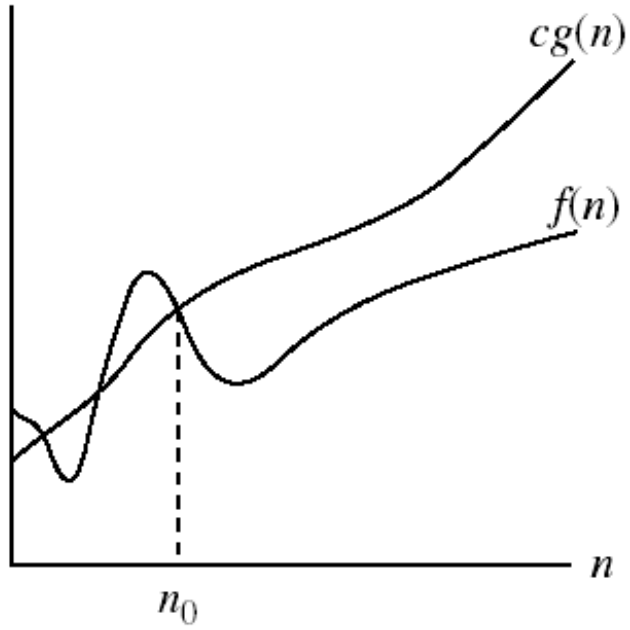
# Example (cont'd)

| ***Algorithm 3*** | ***Cost*** |
|---|---|
| sum = 0; | $c_1$ |
| for(i=0; i<N; i++) | $c_2$ |
| for(j=0; j<N; j++) | $c_2$ |
| sum += arr[i][j]; | $c_3$ |

$$c_1 + c_2 \times (N+1) + c_2 \times N \times (N+1) + c_3 \times N^2 = O(N^2)$$

# Big-O Notation

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\}.$$



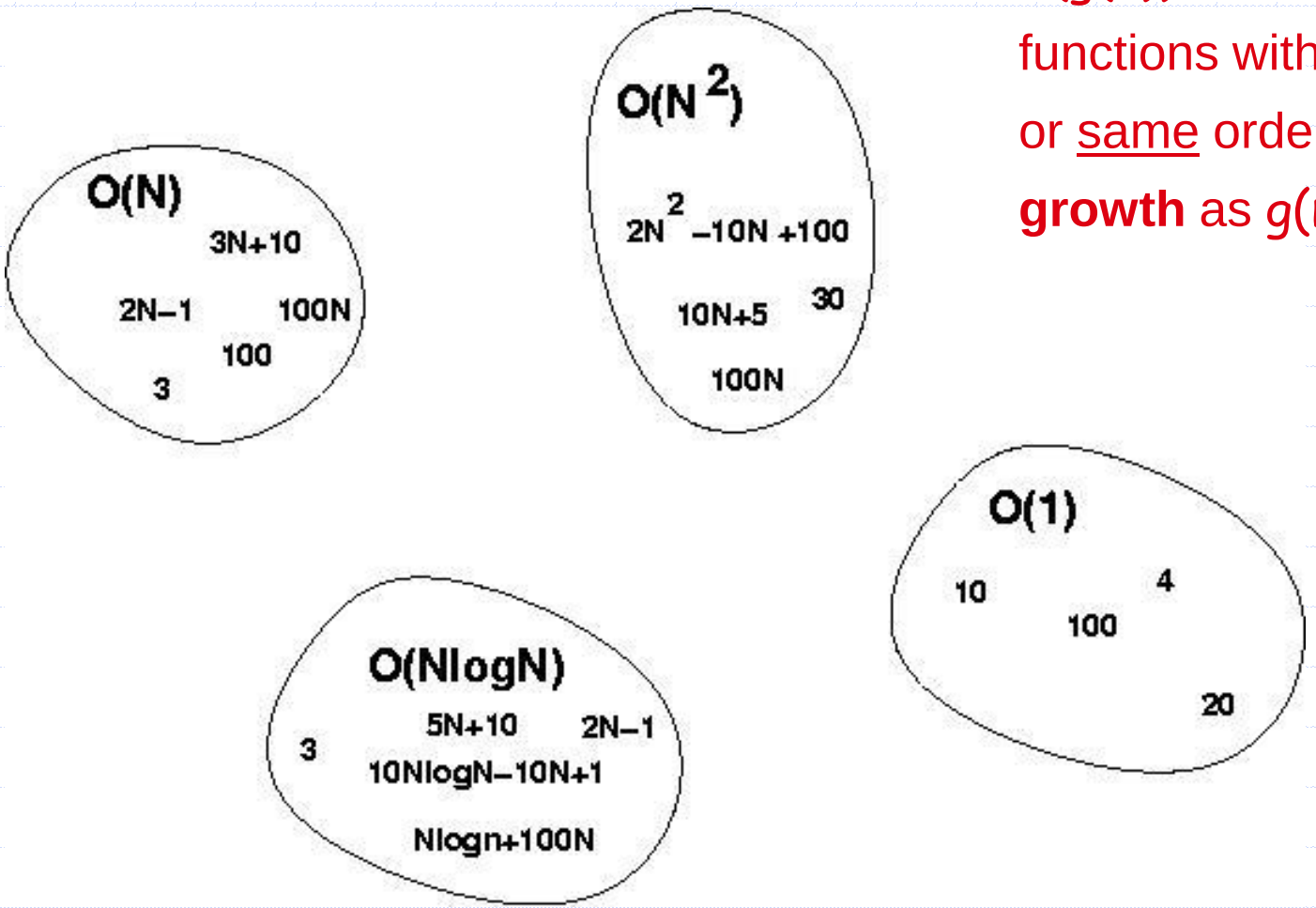

$O(g(n))$ is the set of functions with smaller or same order of **growth** as $g(n)$

$g(n)$ is an ***asymptotic upper bound*** for $f(n)$.

# Big-O Notation

$$f = O(g) \leftrightarrow \exists\, C, n_o > 0$$
$$\ni f(n) \leq Cg(n), \forall\, n \geq n_o$$

# Big-O Visualization




$O(g(n))$ is the set of functions with smaller or same order of **growth** as $g(n)$

# More Examples

- Show that $30n+8$ is $O(n)$.

  Show $\exists c, n_0$: $30n+8 \leq cn$, $\forall n > n_0$

  $30n+8 \leq 30n+8n$

  $30n+8 \leq 38n$

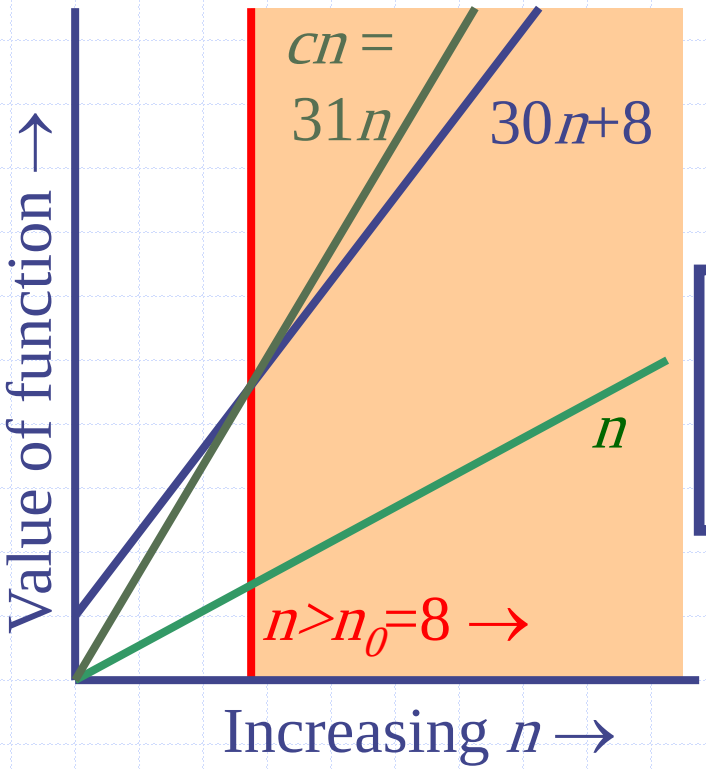
  C=38 for all $n \geq 1$

  $n_0 = 1$

- Note:
  - $f(n) = 30n+8 = O(n)$ (the best is the closest)
  - $f(n) = 30n+8 = O(n^2)$
  - $f(n) = 30n+8 = O(n^3)$
  - $f(n) = 30n+8 \neq O(\log n)$

# Big-O example, graphically

- Note $30n+8$ isn't less than $n$ *anywhere* ($n>0$).
- It isn't even less than $31n$ *everywhere*.
- But it *is* less than $31n$ everywhere to the right of $n=8$.




$30n+8 \in O(n)$

# No Uniqueness

- There is no unique set of values for $n_0$ and $c$ in proving the asymptotic bounds
- Prove that $100n + 5 = O(n^2)$
  - $100n + 5 \leq 100n + n = 101n \leq 101n^2$

    for all $n \geq 5$

    $n_0 = 5$ and $c = 101$ is a solution
  - $100n + 5 \leq 100n + 5n = 105n \leq 105n^2$

    for all $n \geq 1$

    $n_0 = 1$ and $c = 105$ is also a solution

Must find **SOME** constants c and $n_0$ that satisfy the asymptotic notation relation

# Examples

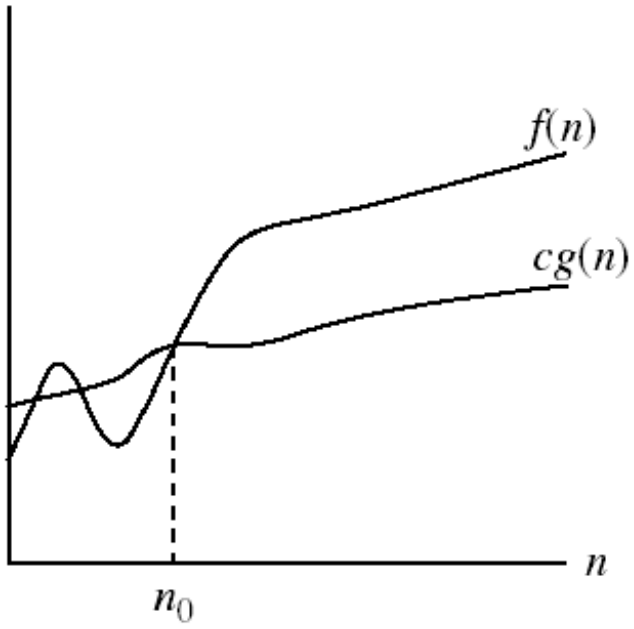
- $2n^2 = O(n^3)$: $2n^2 \leq cn^3 \Rightarrow 2 \leq cn \Rightarrow c = 1$ and $n_0 = 2$
- $n^2 = O(n^2)$: $n^2 \leq cn^2 \Rightarrow c \geq 1 \Rightarrow c = 1$ and $n_0 = 1$
- $1000n^2+1000n = O(n^2)$:

$1000n^2+1000n \leq 1000n^2+ n^2 =1001n^2 \Rightarrow c=1001$ and $n_0 = 1000$

- $n = O(n^2)$: $n \leq cn^2 \Rightarrow cn \geq 1 \Rightarrow c = 1$ and $n_0 = 1$

# Big- Ω Notation

$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0\} .$$




Ω(*g*(n)) is the set of functions with larger or same order of **growth** as *g*(n)

$g(n)$ is an ***asymptotic lower bound*** for $f(n)$.

# Big- $\Omega$ Notation

$$f = \Omega(g) \leftrightarrow \exists\, C, n_o > 0$$
$$\ni f(n) \geq Cg(n), \forall\, n \geq n_o$$

# Examples

- $5n^2 = \Omega(n)$

  $\exists\ c, n_0$ such that: $0 \le cn \le 5n^2 \Rightarrow cn \le 5n^2 \Rightarrow c = 1$ and $n_0 = 1$

- $100n + 5 \neq \Omega(n^2)$

  $\exists\ c, n_0$ such that: $0 \le cn^2 \le 100n + 5$

  $100n + 5 \le 100n + 5n\ (\forall\ n \ge 1) = 105n$

  $cn^2 \le 105n \Rightarrow n(cn - 105) \le 0$

  Since n is positive $\Rightarrow cn - 105 \le 0 \Rightarrow n \le 105/c$

  $\Rightarrow$ contradiction: *n* cannot be smaller than a constant

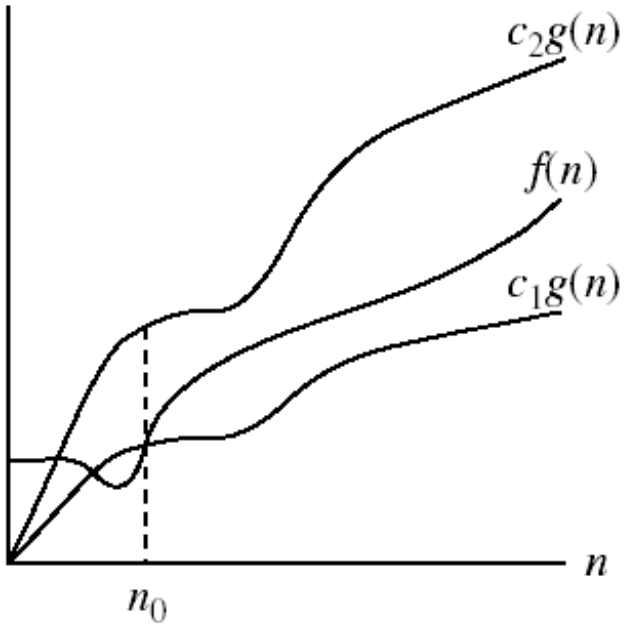
- $n = \Omega(2n)$, $n^3 = \Omega(n^2)$, $n = \Omega(\log n)$

# Examples

- Note:
  - f(n) = 30*n*+8 = $\Omega(n)$. (the best is the closest)
  - f(n) = 30n+8 = $\Omega(\log n)$.
  - f(n) = 30n+8 $\neq$ $\Omega(n^2)$.

# Big- $\Theta$ Notation

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0\} .$$




$\Theta(g(n))$ is the set of functions with the same order of **growth** as $g(n)$

$g(n)$ is an ***asymptotically tight bound*** for $f(n)$.

# Big- $\Theta$ Notation

$$f = \Theta(g) \leftrightarrow \exists\, C_1, C_2, n_o > 0$$
$$\ni\, C_1 g(n) \leq f(n) \leq C_2 g(n), \forall\, n \geq n_o$$

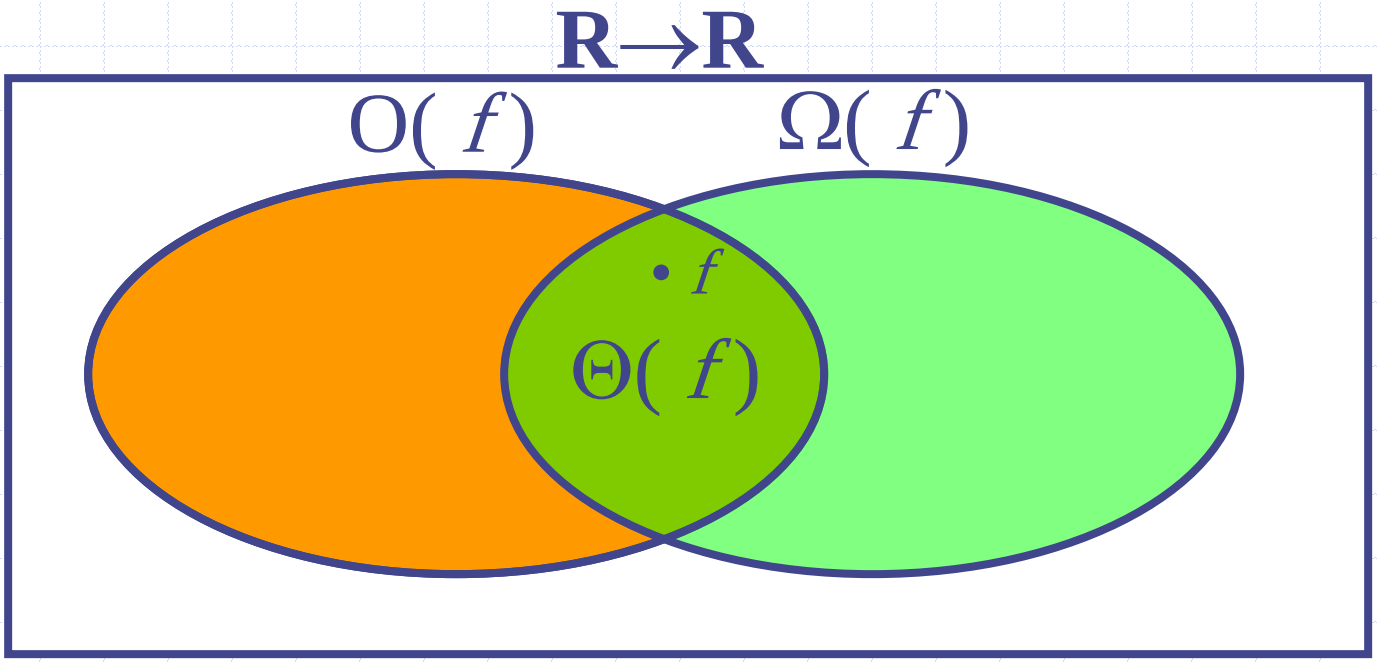
$$f = \Theta(g) \leftrightarrow f = O(g) \text{ and } f = \Omega(g)$$

# Examples

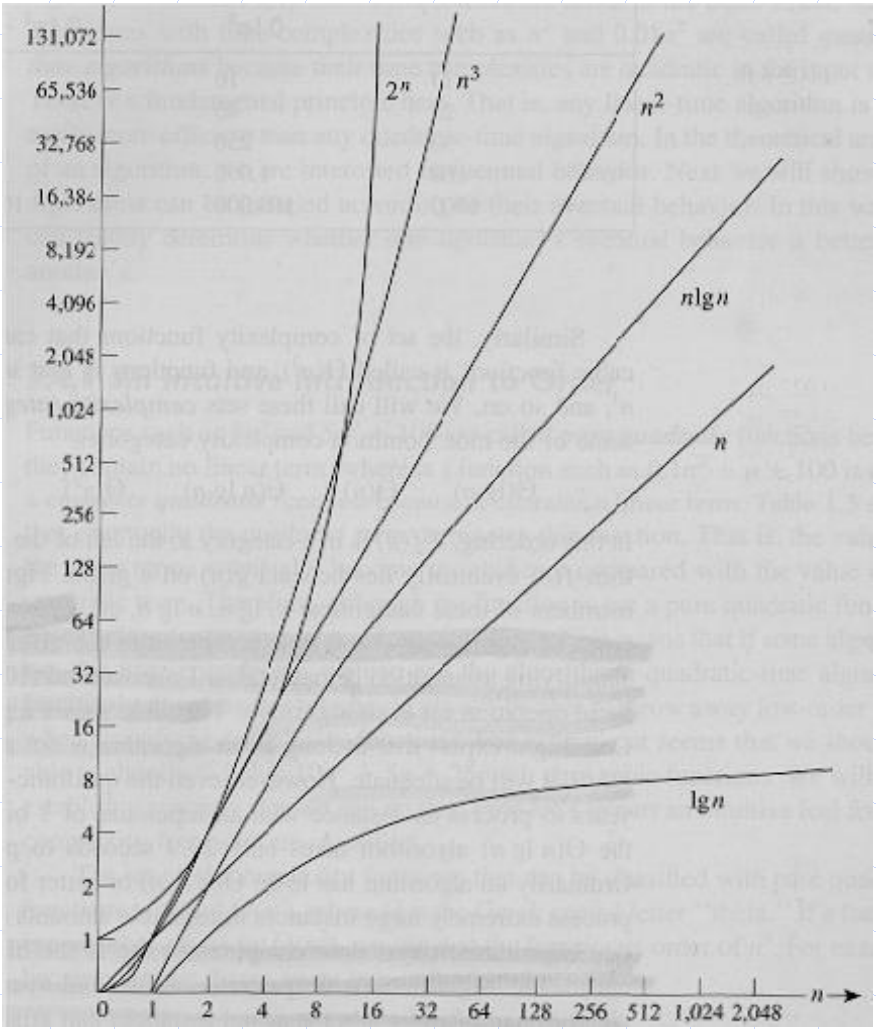
- $f(n) = 30n + 8 = \Theta(n)$
- means: $c_1 n \leq 30n + 8 \leq c_2 n$
  - For example: $c_1 = 1$ and $c_2 = 38$
- **Note:**
  - $f(n) = 30n + 8 = \Theta(n)$.
  - $f(n) = 30n + 8 \neq \Theta(\log n)$.
  - $f(n) = 30n + 8 \neq \Theta(n^2)$.

# Relations Between Different Sets

- Subset relations between order-of-growth sets.



# Common orders of magnitude



# Common orders of magnitude

**Table 1.4** Execution times for algorithms with the given time complexities

| $n$ | $f(n) = \lg n$ | $f(n) = n$ | $f(n) = n \lg n$ | $f(n) = n^2$ | $f(n) = n^3$ | $f(n) = 2^n$ |
|---|---|---|---|---|---|---|
| 10 | 0.003 $\mu$s* | 0.01 $\mu$s | 0.033 $\mu$s | 0.1 $\mu$s | 1 $\mu$s | 1 $\mu$s |
| 20 | 0.004 $\mu$s | 0.02 $\mu$s | 0.086 $\mu$s | 0.4 $\mu$s | 8 $\mu$s | 1 ms† |
| 30 | 0.005 $\mu$s | 0.03 $\mu$s | 0.147 $\mu$s | 0.9 $\mu$s | 27 $\mu$s | 1 s |
| 40 | 0.005 $\mu$s | 0.04 $\mu$s | 0.213 $\mu$s | 1.6 $\mu$s | 64 $\mu$s | 18.3 min |
| 50 | 0.006 $\mu$s | 0.05 $\mu$s | 0.282 $\mu$s | 2.5 $\mu$s | 125 $\mu$s | 13 days |
| $10^2$ | 0.007 $\mu$s | 0.10 $\mu$s | 0.664 $\mu$s | 10 $\mu$s | 1 ms | $4 \times 10^{15}$ years |
| $10^3$ | 0.010 $\mu$s | 1.00 $\mu$s | 9.966 $\mu$s | 1 ms | 1 s | |
| $10^4$ | 0.013 $\mu$s | 10 $\mu$s | 130 $\mu$s | 100 ms | 16.7 min | |
| $10^5$ | 0.017 $\mu$s | 0.10 ms | 1.67 ms | 10 s | 11.6 days | |
| $10^6$ | 0.020 $\mu$s | 1 ms | 19.93 ms | 16.7 min | 31.7 years | |
| $10^7$ | 0.023 $\mu$s | 0.01 s | 0.23 s | 1.16 days | 31,709 years | |
| $10^8$ | 0.027 $\mu$s | 0.10 s | 2.66 s | 115.7 days | $3.17 \times 10^7$ years | |
| $10^9$ | 0.030 $\mu$s | 1 s | 29.90 s | 31.7 years | | |

*1 $\mu$s = $10^{-6}$ second.
†1 ms = $10^{-3}$ second.

# Hierarchy of Functions

| | | |
|---|---|---|
| constant | 1 | |
| log log | log log n | |
| logarithmic | log n | |
| poly-logarithmic | $\log^k n$ | constant integer k>1 |
| sub-linear | $n^\varepsilon$ | Constant 0<ε<1 |
| linear | n | |
| above-linear | n log n | |
| quadratic | $n^2$ | |
| cubic | $n^3$ | |
| polynomial | $n^k$ | constant integer k>1 |
| super-polynomial | $n^{\log n}$ | |
| exponential | $2^n$ | |
| factorial | n! | |
| super-exponential | $2^{2^{\cdots 2}}$ | n powers |

# o-notation (Small o)

- $o(g(n)) = \{ f(n)$: for any positive constant c, there exists a positive constant $n_0$ such that $0 \leq f(n) < cg(n)$ for all $n \geq n_0 \}$

  - Denotes an upper bound that is not asymptotically tight
    - $2n^2 = O(n^2) \rightarrow$ asymptotically tight
    - $2n = O(n^2) \rightarrow$ not asymptotically tight
    - $2n = o(n^2)$ but $2n^2 \neq o(n^2)$

# ω-notation (Small Omega)

- $\omega(g(n)) = \{ f(n)$: for any positive constant c, there exists a positive constant $n_0$ such that $0 \leq cg(n) < f(n)$ for all $n \geq n_0\}$
  - Denotes a lower bound that is not asymptotically tight
    - $n^2/2 = \omega(n)$ but $n^2/2 \neq \omega(n^2)$

# Theorems

- In little o, the function f (n) becomes insignificant relative to g(n) as n approaches infinity:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$ means f(n) = *o(g(n))*

- For little omega ,

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$$

means f(n) = $\omega\,(g(n))$

# Logarithms and properties

In algorithm analysis we often use the notation "log n" without specifying the base.

Binary logarithm $\lg n = \log_2 n$

Natural logarithm $\ln n = \log_e n$

$\lg^k n = (\lg n)^k$

$\lg \lg n = \lg(\lg n)$

$\log x^y = y \log x$

$\log xy = \log x + \log y$

$\log \frac{x}{y} = \log x - \log y$

$a^{\log_b x} = x^{\log_b a}$

$\log_b x = \frac{\log_a x}{\log_a b}$

# Theorems

- For any polynomial function $p(n) = \sum_{i=0}^{d} a_i n^i$ where $a_i$ are constants and $a_d > 0$, then $p(n) = \theta(n^d)$.'

- If $f_1(n) = O(g_1(n))$ and if $f_2(n) = O(g_2(n))$ then:
  - $f_1(n) + f_2(n) = O(\max(g_1, g_2))$ and
  - $f_1(n).f_2(n) = O(g_1.g_2)$

# Properties of Asymptotic Notations

## 1. Transitivity

$$
\begin{aligned}
f(n) &= \Theta(g(n)) \text{ and } g(n) = \Theta(h(n)) \text{ imply } f(n) = \Theta(h(n)) ,\\
f(n) &= O(g(n)) \text{ and } g(n) = O(h(n)) \text{ imply } f(n) = O(h(n)) ,\\
f(n) &= \Omega(g(n)) \text{ and } g(n) = \Omega(h(n)) \text{ imply } f(n) = \Omega(h(n)) ,\\
f(n) &= o(g(n)) \text{ and } g(n) = o(h(n)) \text{ imply } f(n) = o(h(n)) ,\\
f(n) &= \omega(g(n)) \text{ and } g(n) = \omega(h(n)) \text{ imply } f(n) = \omega(h(n)) .
\end{aligned}
$$

## 2. Reflexivity

$$
\begin{aligned}
f(n) &= \Theta(f(n)) ,\\
f(n) &= O(f(n)) ,\\
f(n) &= \Omega(f(n)) .
\end{aligned}
$$

## 3. Symmetry

$$f(n) = \Theta(g(n)) \text{ if and only if } g(n) = \Theta(f(n)) .$$

## 4. Transport Symmetry

$$
\begin{aligned}
f(n) &= O(g(n)) \text{ if and only if } g(n) = \Omega(f(n)) ,\\
f(n) &= o(g(n)) \text{ if and only if } g(n) = \omega(f(n)) .
\end{aligned}
$$

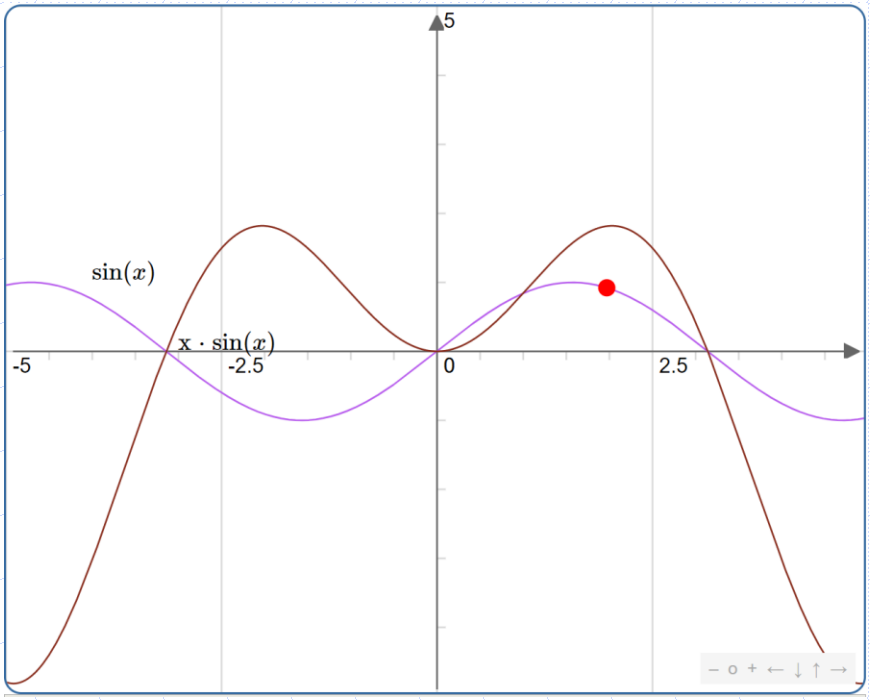
# Examples

- Let $f(n) = n$, $g(n)=n^2$, $h(n) = n^3$
- Addition:
  - $f(n) + g(n) = n + n^2$
  - $= O(n^2)$ $= O(\max (f(n) , g(n) )$
- Multiplication:
  - $f(n) * g(n) = n * n^2$
  - $= O(n^3)$ $= O(f(n) * g(n))$
- Transitive:
  - $f(n) = O(n^2)$ , $g(n) = O(n^3)$
  - Then: $f(n) = O(n^3)$

# Properties of Asymptotic Notations

- **Note:**

- For any two functions f(n) and g(n), it may be the case that neither f(n) = O (g(n))  nor f(n) = Ω (g(n))



# More Examples

- For each of the following pairs of functions, either f(n) is O(g(n)), f(n) is Ω(g(n)), or f(n) = Θ(g(n)). Determine which relationship is correct.

| | |
|---|---|
| $f(n) = \log n^2$; $g(n) = \log n + 5$ | $f(n) = \Theta(g(n))$ |
| $f(n) = n$; $g(n) = \log n^2$ | $f(n) = \Omega(g(n))$ |
| $f(n) = \log \log n$; $g(n) = \log n$ | $f(n) = O(g(n))$ |
| $f(n) = n$; $g(n) = \log^2 n$ | $f(n) = \Omega(g(n))$ |
| $f(n) = n \log n + n$; $g(n) = \log n$ | $f(n) = \Omega(g(n))$ |
| $f(n) = 10$; $g(n) = \log 10$ | $f(n) = \Theta(g(n))$ |
| $f(n) = 2^n$; $g(n) = 10n^2$ | $f(n) = \Omega(g(n))$ |
| $f(n) = 2^n$; $g(n) = 3^n$ | $f(n) = O(g(n))$ |